Reading: cookies vs localStorage, where tokens should live

Objectives

- Compare an HttpOnly cookie with localStorage for script access and exposure to cross-site scripting
- Explain what the Secure, HttpOnly and SameSite cookie attributes do

Teaching

An attacker has managed to run script in your page, through a cross-site scripting bug. Your app keeps its login token in one of two places. Predict what that script can read.

`localStorage` is readable by any script on the origin and is never sent automatically. A cookie is sent automatically with matching requests. `HttpOnly` hides it from script, `Secure` sends it over HTTPS only, and `SameSite` limits sending on cross-site requests.

OWASP advises against keeping session identifiers in `localStorage`. An `HttpOnly` cookie is not a cure for XSS, though: injected script can still send requests as the user.

Each attribute answers one question. HttpOnly: can page script read it? Secure: can it travel over plain HTTP? SameSite: is it sent when another site starts the request? A strong session cookie usually sets all three.

You are building a browser client for your Spring Boot API, on one site you control. It handles real accounts, so a stolen long-lived token would be serious.

Recap

Script on your origin can read `localStorage`, so XSS can copy a token kept there. `HttpOnly` hides a cookie from script, `Secure` keeps it off plain HTTP, and `SameSite` limits cross-site sending. Automatic sending brings CSRF risk, so protect against that too. No storage choice removes the need to prevent XSS.

Cheat sheet

What can the injected script read?

- The `localStorage` token, but not the `HttpOnly` cookie: correct. All script on the origin can read `localStorage`. `HttpOnly` hides a cookie from script entirely, while the browser still sends it with requests.
- Both, because script can read anything in the browser: not correct. That is why `HttpOnly` exists: `document.cookie` does not show those cookies.
- Neither, because both are private to your site: not correct. They are private from other origins. Injected script runs as your origin.
- The cookie, but not `localStorage`: not correct. That is reversed.

Where to act: The Cookie attributes section.

Next step: Change one attribute at a time and rerun the same attack, so you can tie the result to that attribute.

Script reads a localStorage token, and can send it anywhere. HttpOnly hides a cookie from script, but injected script can still call the API with it. A cross-site POST carries a SameSite=None cookie but not a Strict one. Secure keeps a cookie off plain HTTP.

Where to act: The row about another site.

Next step: Look at the three questions on this page, and find the one that mentions who started the request.

- Keeps the cookie out of `document.cookie`: HttpOnly. Script cannot read it, but the browser still sends it.
- Sends the cookie only on HTTPS requests: Secure. It stops the cookie being sent where others can read the traffic.
- Limits sending when a request starts from another site: SameSite. This is the main browser-side defense against cross-site request forgery.
- Reduces what stolen-token XSS can get, but does not stop XSS itself: HttpOnly. Injected script can still act through your page, though it cannot copy the cookie.

Which design fits best?

- An `HttpOnly`, `Secure`, `SameSite` session cookie, with CSRF protection and work to prevent XSS: correct. Script cannot copy the session, and the attributes reduce cross-site use. XSS can still act as the user while it runs, so preventing it remains essential.

- A token in `localStorage`, because it cannot be used in CSRF: not correct. True about CSRF, but any injected script can copy it and use it elsewhere, and OWASP advises against it for sessions.
- A readable cookie with no attributes, because it is simple: not correct. This has both weaknesses: it can be stolen by script and sent cross-site.
- A token in a global JavaScript variable, because that is private: not correct. Any script on the page can read it too, and it is lost on reload.

```
Set-Cookie: session=abc; HttpOnly; Secure; SameSite=Lax; Path=/
```

Sources

- OWASP: HTML5 Security Cheat Sheet, retrieved 2026-11-04
- OWASP: Session Management Cheat Sheet, retrieved 2026-11-04
- MDN: Set-Cookie, retrieved 2026-11-04
- WHATWG HTML: Web storage, retrieved 2026-11-04
- RFC 6265: HTTP State Management, retrieved 2026-11-04

First draft, retrieved 2026-11-04. These claims have not had independent review yet, so treat them as unverified until that review is recorded.

Static interaction alternatives

predict

Static alternative

What can the injected script read?

- The `localStorage` token, but not the `HttpOnly` cookie: correct. All script on the origin can read `localStorage`. `HttpOnly` hides a cookie from script entirely, while the browser still sends it with requests.
- Both, because script can read anything in the browser: not correct. That is why `HttpOnly` exists: `document.cookie` does not show those cookies.
- Neither, because both are private to your site: not correct. They are private from other origins. Injected script runs as your origin.
- The cookie, but not `localStorage`: not correct. That is reversed.

explore

Static alternative

Script reads a localStorage token, and can send it anywhere. HttpOnly hides a cookie from script, but injected script can still call the API with it. A cross-site POST carries a SameSite=None cookie but not a Strict one. Secure keeps a cookie off plain HTTP.

explore

Static alternative

- Keeps the cookie out of `document.cookie`: HttpOnly. Script cannot read it, but the browser still sends it.
- Sends the cookie only on HTTPS requests: Secure. It stops the cookie being sent where others can read the traffic.
- Limits sending when a request starts from another site: SameSite. This is the main browser-side defense against cross-site request forgery.
- Reduces what stolen-token XSS can get, but does not stop XSS itself: HttpOnly. Injected script can still act through your page, though it cannot copy the cookie.

mcq

Static alternative

Which design fits best?

- An `HttpOnly`, `Secure`, `SameSite` session cookie, with CSRF protection and work to prevent XSS: correct. Script cannot copy the session, and the attributes reduce cross-site use. XSS can still act as the user while it runs, so preventing it remains essential.
- A token in `localStorage`, because it cannot be used in CSRF: not correct. True about CSRF, but any injected script can copy it and use it elsewhere, and OWASP advises against it for sessions.
- A readable cookie with no attributes, because it is simple: not correct. This has both weaknesses: it can be stolen by script and sent cross-site.
- A token in a global JavaScript variable, because that is private: not correct. Any script on the page can read it too, and it is lost on reload.